

LAB3-22 · 스텝모터 위상 제어

전자전기컴퓨터설계실험II · FPGA 응용 실습

clock-enable마다 4상 코일 패턴을 이동하여 정·역회전과 정지를 제어한다.

Vivado 2026.1 · FPGA Part xc7s75fgga484-1

작성일 2026. 09. 21. · 이해리

목차

동작과 검증 목표

v2.0.2 프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code·Icarus 시뮬레이션

Vivado XSim·비트스트림

보드 실험과 레포트

이번 회로의 목표

clock-enable마다 4상 코일 패턴을 이동하여 정·역회전과 정지를 제어한다.

핵심 개념: clock-enable, 입력 동기화, 4상 시퀀스, 방향 전환

기능 TB: tb_stepper

예상 PASS: LAB3_STEPPER_PASS checks=8

입출력과 보드 연결

clk_50mhz B6 · rst_p K4 · enable N8 · direction N4 · stepmotor[3:0]
Y20/Y22/AA20/AA21

모든 순차 로직은 MAIN CLOCK F의 50 MHz만 사용한다. 느린 동작은 내부 clock-enable로 만든다.

XDC의 포트 이름은 top module 선언과 글자·대소문자까지 같아야 한다.

TB가 자동으로 확인하는 범위

정방향 한 주기, 역방향 두 단계, enable=0 유지 조건을 가속된 step rate로 검사한다.

시뮬레이션 파라미터는 실제 동작의 순서와 경계 조건을 유지하면서 시간을 줄인다.

PASS는 디지털 기능 검사 결과다. 핀 배선, 전기 규격, 실제 부품의 물리 동작은 보드 단계에서 확인한다.

v2.0.2 템플릿을 새 이름으로 clone

PowerShell에서 한 줄씩 실행한다.

```
1 git clone --branch v2.0.2 https://github.com/Glaysia/fpga-lab-template.git lab3_22_stepper
2 cd lab3_22_stepper
3 git switch -c main
4 code FPGA.code-workspace
```

태그 clone 직후 detached HEAD 안내는 정상이다. git switch -c main으로 자신의 이력을 시작한다.

빈 템플릿에서 만들 파일

경로	파일
src/	lab3_stepper.v
sim/	tb_stepper.sv
constraints/	lab3_stepper.xdc
프로젝트 루트	simulation.json

기존 빈 파일은 이름을 바꾸고, 필요한 추가 RTL은 src에 새로 만든다.

simulation.json 확인

sources에는 모든 RTL 파일을 의존 순서에 맞춰 넣는다.

testbench는 sim/tb_stepper.sv, simulation_top은 tb_stepper다.

설계 top lab3_stepper은 Vivado 합성에서 선택한다. TB module을 합성 top 으로 선택하지 않는다.

코드를 읽고 입력하는 기준

다음 코드 화면은 줄 번호와 파일명이 원본 프로젝트와 연결되어 있다.

RTL은 합성 가능한 회로, TB는 입력 자극과 기대값 비교, XDC는 핀과 20 ns 클록 제약을 담당한다.

코드를 붙여 넣는 것으로 끝내지 말고 counter 폭, 파라미터 계산, 리셋 우선순위, 출력 시점을 설명할 수 있어야 한다.

simulation.json · 1-5행

```
1 {  
2   "sources": ["src/lab3_stepper.v"],  
3   "testbench": "sim/tb_stepper.sv",  
4   "simulation_top": "tb_stepper"  
5 }
```

lab3_stepper.v · 1-18행

```
1 `timescale 1ns/1ps
2 module lab3_stepper #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer STEP_HZ = 100
5 ) (
6     input wire clk_50mhz,
7     input wire rst_p,
8     input wire enable,
9     input wire direction,
10    output reg [3:0] stepmotor
11 );
12    localparam integer STEP_CYCLES = CLK_HZ / STEP_HZ;
13    localparam integer COUNT_WIDTH = (STEP_CYCLES < 2) ? 1 : $clog2(STEP_CYCLES);
14    reg [COUNT_WIDTH-1:0] count;
15    reg [1:0] state;
16    (* ASYNC_REG = "TRUE" *) reg enable_meta, enable_sync;
17    (* ASYNC_REG = "TRUE" *) reg direction_meta, direction_sync;
```

lab3_stepper.v · 19-36행

```
19  always @(posedge clk_50mhz or posedge rst_p) begin
20      if (rst_p) begin
21          enable_meta <= 0; enable_sync <= 0;
22          direction_meta <= 0; direction_sync <= 0;
23      end else begin
24          enable_meta <= enable; enable_sync <= enable_meta;
25          direction_meta <= direction; direction_sync <= direction_meta;
26      end
27  end
28
29  always @(posedge clk_50mhz or posedge rst_p) begin
30      if (rst_p) begin
31          count <= 0;
32          state <= 0;
33      end else if (!enable_sync) begin
34          count <= 0;
35      end else if (count == STEP_CYCLES - 1) begin
36          count <= 0;
```

lab3_stepper.v · 37-51행

```
37     state <= direction_sync ? state - 1'b1 : state + 1'b1;
38   end else begin
39     count <= count + 1'b1;
40   end
41 end
42
43 always @* begin
44   case (state)
45     2'd0: stepmotor = 4'b0011;
46     2'd1: stepmotor = 4'b0110;
47     2'd2: stepmotor = 4'b1100;
48     default: stepmotor = 4'b1001;
49   endcase
50 end
51 endmodule
```

tb_stepper.sv · 1-18행

```
1 `timescale 1ns/1ps
2 module tb_stepper;
3     reg clk_50mhz=0,rst_p=1,enable=0,direction=0;
4     wire [3:0] stepmotor;
5     integer checks=0;
6     always #10 clk_50mhz=~clk_50mhz;
7     lab3_stepper #(.CLK_HZ(8),.STEP_HZ(2)) dut
8         (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.enable(enable),.direction(direction),.stepmotor(
9             stepmotor));
10    task check_value(input [3:0] value);begin #1;if(stepmotor!=value)$fatal(1,"got=%b expected=%b"
11        ,stepmotor,value);checks++;end endtask
12    initial begin
13        $dumpfile("wave.vcd");$dumpvars(0,tb_stepper);
14        repeat(2)@(posedge clk_50mhz);rst_p=0;check_value(4'b0011);
15        enable=1;wait(dut.enable_sync);wait(dut.state==1);check_value(4'b0110);
16        wait(dut.state==2);check_value(4'b1100);
17        wait(dut.state==3);check_value(4'b1001);
18        wait(dut.state==0);check_value(4'b0011);
19        direction=1;wait(dut.direction_sync);wait(dut.state==3);check_value(4'b1001);
20        wait(dut.state==2);check_value(4'b1100);
```

tb_stepper.sv · 19–23행

```
19     enable=0;wait(!dut.enable_sync);repeat(12)@(posedge clk_50mhz);check_value(4'b1100);
20     $display("LAB3_STEPPER_PASS checks=%0d",checks);$finish;
21     end
22     initial begin #5000;$fatal(1,"timeout");end
23 endmodule
```

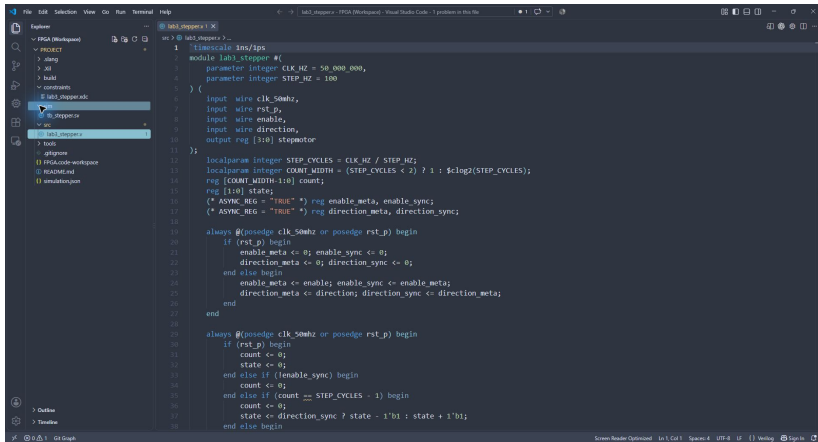
lab3_stepper.xdc · 1-12행

```
1 # Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.  
2 set_property PACKAGE_PIN B6 [get_ports clk_50mhz]  
3 set_property PACKAGE_PIN K4 [get_ports rst_p]  
4 set_property PACKAGE_PIN N8 [get_ports enable]  
5 set_property PACKAGE_PIN N4 [get_ports direction]  
6 set_property PACKAGE_PIN Y20 [get_ports {stepmotor[3]}]  
7 set_property PACKAGE_PIN Y22 [get_ports {stepmotor[2]}]  
8 set_property PACKAGE_PIN AA20 [get_ports {stepmotor[1]}]  
9 set_property PACKAGE_PIN AA21 [get_ports {stepmotor[0]}]  
10 set_property IOSTANDARD LVCMOS33 [get_ports *]  
11 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]  
12 set_false_path -from [get_ports {rst_p enable direction}]
```

VS Code에서 Icarus 실행

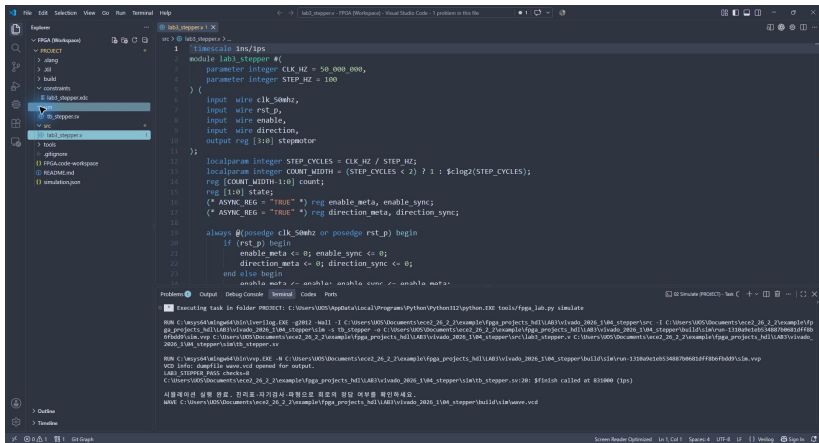
1. File → Save All.
2. Terminal → Run Task... → 01 Check tools.
3. Git·Python·iverilog·vvp 경로와 버전을 확인한다.
4. Terminal → Run Task... → 02 Simulate.
5. LAB3_STEPPER_PASS checks=8와 종료 시각을 기록한다.
6. 03 Open waveform으로 build/sim/wave.vcd를 연다.

VS Code · 프로젝트와 파일



04_stepper-vscode-workspace

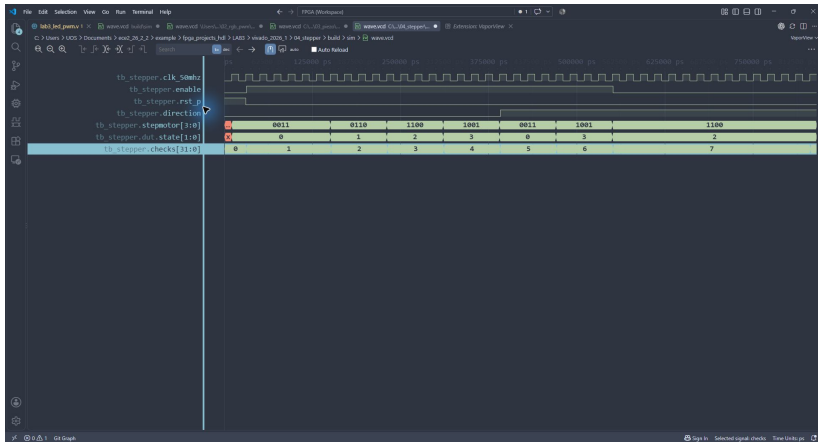
VS Code · 시뮬레이션 PASS



04 stepper-vscode-pass

목차

VS Code · 파형



04_stepper-vscode-wave

파형에서 설명할 내용

정방향 한 주기, 역방향 두 단계, enable=0 유지 조건을 가속된 step rate로 검사한다.

입력을 바꾼 시점, 유효 상승 에지, 상태 갱신 직후를 구분한다.

파라미터를 줄인 TB 시간과 실제 50 MHz 보드 시간을 혼동하지 않는다.

수정 실험

STEP_HZ를 절반으로 바꾸어 단계 간격을 계산하고 회전 속도를 비교한 뒤 복구한다.

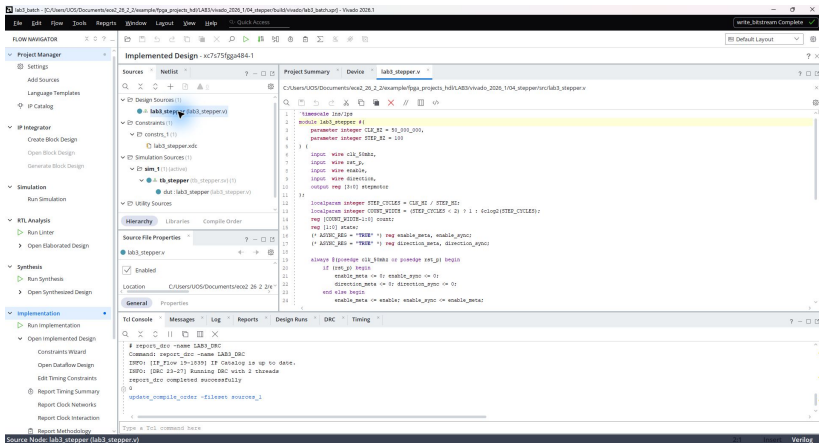
정상 PASS와 파형을 먼저 보관한 뒤 한 항목만 수정한다. 예상값, 실제 FAIL 또는 변화, 복구 PASS를 3행 표로 기록한다.

TB의 기대값을 RTL 오류에 맞춰 바꾸지 않는다.

Vivado 2026.1 프로젝트 생성

1. Create Project → RTL Project → Do not specify sources at this time.
2. Part에 xc7s75fgga484-1을 정확히 선택한다.
3. Add Sources에 src의 RTL 전부, Add Simulation Sources에 TB를 추가한다.
4. Add or Create Constraints에 XDC를 추가한다.
5. Design Sources의 top을 lab3_stepper으로 설정한다.

Vivado · 소스와 top



04_stepper-vivado-sources

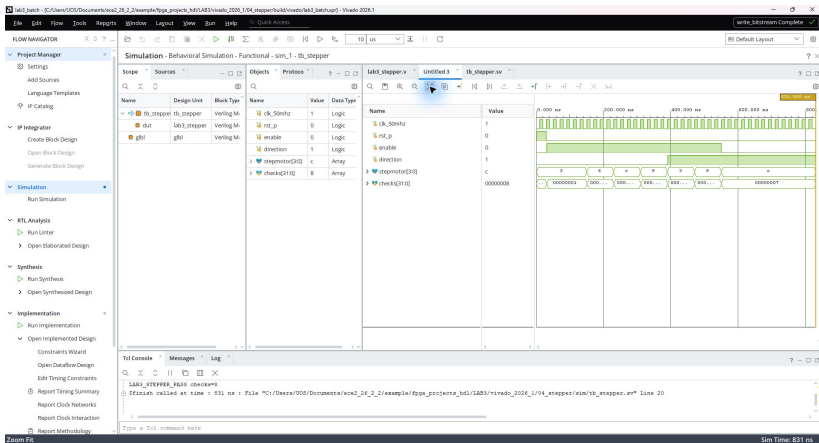
Vivado XSim

Flow Navigator → Simulation → Run Behavioral Simulation.

Tcl Console의 LAB3_STEPPER_PASS checks=8를 확인한다. Objects/Wave에서 입력과 출력을 추가하고 Zoom Fit으로 전체 실행을 본다.

Icarus와 XSim은 같은 RTL·TB를 실행해야 한다. 서로 다른 사본을 등록하지 않는다.

Vivado · XSim PASS와 파형



04_stepper-xsim-pass

핀과 클록 제약 확인

Open Elaborated Design 또는 I/O Planning에서 top port와 PACKAGE_PIN을 대조한다.

Reports → Timing → Report Clocks에서 clk_50mhz 주기 20.000 ns를 확인한다.

XDC를 저장한 뒤 Run Synthesis → Run Implementation → Generate Bitstream 순서로 실행한다.

Vivado · 구현과 bit 생성

The screenshot shows the Vivado IDE interface with the following components:

- Project Manager (Left):** Lists project files and folders, including 'lab3_stepper', 'Nets (1)', and 'Leaf Cells (7)'. The 'Implementation' section is expanded, showing options like 'Run Implementation', 'Open Implemented Design', 'Constraints Wizard', 'Open Dataflow Design', 'Edit Timing Constraints', 'Report Timing Summary', 'Report Clock Networks', 'Report Clock Interaction', and 'Report Methodology'.
- Implemented Design - xc7z75fpga484-1:**
 - Sources:** Lists 'lab3_stepper', 'Nets (1)', and 'Leaf Cells (7)'.
 - Project Summary:**
 - Synthesis:** Status is 'Complete'. Messages: 'No errors or warnings'. Part: 'xc7z75fpga484-1'. Strategy: 'Vivado Synthesis Defaults'. Report Strategy: 'Vivado Synthesis Default Reports'. Incremental synthesis: 'None'.
 - Implementation:** Status is 'Complete'. Messages: 'No errors or warnings'. Part: 'xc7z75fpga484-1'. Strategy: 'Vivado Implementation Defaults'. Report Strategy: 'Vivado Implementation Default Reports'. Incremental implementation: 'None'.
 - Timing:** Summary shows '1 warning' and 'Implemented DRC Report'. Details include: Worst Negative Slack (WNS): 16.252 ns, Total Negative Slack (TNS): 0 ns, Number of Failing Endpoints: 0, Total Number of Endpoints: 23. Links for 'Implemented DRC Report' and 'Implemented Timing Report' are provided.
 - Utilization:** Tabs for 'Post Synthesis', 'Post Implementation', and 'Power' are visible.
- Tcl Console:** Shows the following commands and output:


```
# report_dir -name LAB3_DRC
Command: report_dir -name LAB3_DRC
INFO: [Tcl_Flow 19-1039] IF Catalog is up to date.
INFO: [DRC 23-27] Running DRC with 2 threads
report_dir completed successfully
0
update_compile_order -fileset sources_1
```

04_stepper-vivado-bit

보드에서 확인

전원을 끈 상태에서 외부 부품 결선과 공통 GND를 먼저 확인한다.

Hardware Manager → Open Target → Auto Connect → Program Device에서
생성한 bit를 기록한다.

드라이버 결선 후 회전 방향·단계 속도·정지 유지와 모터 발열을 관찰한다.

사진에는 보드 전체와 사용 핀이, 영상에는 입력 조작과 출력 변화가 함께
보이게 한다.

실험 전 레포트

1. 동작 원리와 핵심 파라미터 계산을 자기 말로 설명한다.
2. RTL·TB·XDC 각 파일의 역할과 top module을 적는다.
3. TB 자극과 기대 결과를 표로 정리한다.
4. Icarus PASS·파형과 수정/복구 결과를 넣고 경계 조건을 설명한다.
5. 보드에서 확인해야 할 항목과 예상 결과를 적는다.

실험 후 레포트와 제출

1. XSim PASS·파형, DRC·timing·bitstream 결과를 기록한다.
2. 실제 보드 관찰, 계산값과 실측의 차이, 원인을 설명한다.
3. 결선이 보이는 사진과 동작 영상 링크를 첨부한다.
4. 자신의 GitHub 저장소 URL과 최종 commit hash를 적는다.
5. src·sim·constraints·simulation.json만 최종 원본으로 관리하고 생성물은 제외한다.

완료 체크

- ☐ v2.0.2 태그에서 새 프로젝트를 만들었다.
- ☐ RTL·TB·XDC를 직접 작성하고 설명할 수 있다.
- ☐ Icarus와 XSim에서 같은 PASS를 확인했다.
- ☐ 핀·20 ns 클록·DRC·timing을 확인하고 bit를 만들었다.
- ☐ 보드 동작 사진·영상과 GitHub 링크를 정리했다.
- ☐ 실험 전·후 레포트에 결과와 한계를 구분해 기록했다.